Cloud OAuth Apps Configuration Guide

This guide provides step-by-step instructions for obtaining OAuth credentials (Client ID and Client Secret) from various OAuth providers to configure in the Activepieces Cloud OAuth Apps dashboard.

Table of Contents

- Overview
- Prerequisites
- OAuth Provider Setup Guides
  - Slack
  - Google (Gmail, Sheets, Drive, Calendar)
  - HubSpot
  - Microsoft (Outlook, Teams, OneDrive)
  - GitHub
  - GitLab
  - Notion
  - Dropbox
  - Salesforce
  - Snowflake
  - Asana
  - ClickUp
  - Monday.com
  - Trello
  - Jira
  - Linear
  - Figma
  - Zoom
  - Todoist
  - Typeform
  - QuickBooks
  - Intercom
  - Zendesk
  - Mailchimp
  - Shopify
  - Stripe
- Configuring in Activepieces
- App Webhook Configuration (For Triggers)
  - Pieces Requiring App Webhook Configuration
  - Configure Webhook Signing Secret
  - Configure Event Subscriptions
- Troubleshooting
- Supported Pieces Reference

Overview

Cloud OAuth Apps allow you to configure your own OAuth credentials for various integrations. This enables:

- **Simplified connection flow**: Users see a simple "Connect" button instead of entering credentials manually
- **Branding control**: Your app name appears during OAuth authorization
- **Security**: You control and manage your own OAuth credentials

Understanding OAuth Models

OAuth integrations fall into two categories based on how the OAuth app is registered and which users can authenticate:

Model	Description	Examples
Centralized OAuth	One OAuth app works for all users across any account/workspace. Register once, use everywhere.	Slack, Google, Microsoft, HubSpot, GitHub
Org/Account-Specific OAuth	Each organization or account requires its own OAuth app configuration. Apps are isolated per org.	Salesforce, Snowflake

Why This Matters

Centralized OAuth (Most Pieces)

- Create one OAuth app → all users can connect
- Simple setup and maintenance
- Example: One Slack app works for any Slack workspace

Org-Specific OAuth (Salesforce, Snowflake)

- Each org/account needs its own OAuth app created
- Credentials from Org A **will not work** for users in Org B
- More complex for multi-tenant deployments
- See individual piece sections for deployment strategies

> **Quick Reference:** If you're setting up Salesforce or Snowflake, pay special attention to their org-specific requirements.

Understanding OAuth vs App Webhooks

For pieces like Slack, there are **two separate configurations** needed for full functionality:

Configuration	Purpose	Required For
OAuth (Cloud OAuth Apps)	Authenticate users and get access tokens	**Actions** (send message, create channel, etc.)
App Webhooks	Receive real-time events from the service	**Triggers** (new message, reaction added, etc.)

```
┌──────────────────────────────────────────────────────────────┐
│                        OAUTH (This Guide)                    │
│  User → Connect → OAuth Popup → Token Stored → Actions Work  │
└──────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│                      APP WEBHOOKS (See Below)                │
│  Service Event → Webhook URL → Verify Signature → Trigger Fires │
└──────────────────────────────────────────────────────────────┘
```

If you only need Actions (e.g., "Send Slack Message"), configure OAuth only.
If you need Triggers (e.g., "New Slack Message"), configure BOTH OAuth AND App Webhooks.

Prerequisites

Before configuring Cloud OAuth Apps, ensure:

1. **Redirect URL**: Note your Activepieces redirect URL:

   `https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect`

2. **Super Admin Access**: You need Super Admin privileges to access the Cloud OAuth Apps configuration page.
3. **Developer Account**: You'll need a developer account with each OAuth provider you want to configure.

OAuth Provider Setup Guides

Slack

Developer Portal: https://api.slack.com/apps

1. Go to Slack API Apps
2. Click **Create New App** → **From scratch**
3. Enter your app name and select your workspace
4. Navigate to **OAuth & Permissions** in the sidebar
5. Under **Redirect URLs**, add:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Add the following **Bot Token Scopes** (required by Activepieces):
   - `channels:read` - View basic channel info
   - `channels:manage` - Manage channels
   - `channels:history` - View messages in channels
   - `channels:write.invites` - Invite users to channels
   - `chat:write` - Send messages
   - `chat:write.customize` - Send messages with custom username/icon
   - `emoji:read` - View custom emoji
   - `files:read` - View files
   - `files:write` - Upload files
   - `groups:read` - View private channels
   - `groups:write` - Manage private channels
   - `groups:history` - View messages in private channels
   - `groups:write.invites` - Invite users to private channels
   - `im:read` - View direct messages
   - `im:write` - Start direct messages
   - `im:history` - View direct message history
   - `links:read` - View URLs in messages
   - `links:write` - Unfurl URLs
   - `mpim:read` - View group direct messages
   - `mpim:write` - Start group direct messages
   - `mpim:history` - View group DM history
   - `reactions:read` - View reactions
   - `reactions:write` - Add/remove reactions
   - `usergroups:read` - View user groups
   - `users:read` - View users
   - `users:read.email` - View user email addresses
   - `users.profile:read` - View user profiles
7. Add the following **User Token Scopes** (for user-level actions):
   - `search:read` - Search messages
   - `users.profile:write` - Update user profile
   - `reactions:read` - View reactions
   - `reactions:write` - Add reactions
   - `im:history` - View DM history
   - `stars:read` - View starred items
   - `channels:write` - Manage channels
   - `groups:write` - Manage private channels
   - `im:write` - Start DMs
   - `mpim:write` - Start group DMs
   - `channels:write.invites` - Invite to channels
   - `groups:write.invites` - Invite to private channels
   - `channels:history` - View channel history

- `groups:history` - View private channel history
- `chat:write` - Send messages
- `users:read` - View users

8. Navigate to **Basic Information**
9. Under **App Credentials**, find:
   - **Client ID**: Copy this value
   - **Client Secret**: Click "Show" and copy this value
   - **Signing Secret**: Copy this for App Webhooks (triggers)

> **For Slack Triggers**: You also need to configure Event Subscriptions. See the App Webhook Configuration section below.

Google (Gmail, Sheets, Drive, Calendar)

Developer Console: https://console.cloud.google.com/

1. Go to Google Cloud Console
2. Create a new project or select existing one
3. Navigate to **APIs & Services** → **Library** and enable required APIs:
   - **Gmail API** (for Gmail piece)
   - **Google Sheets API** (for Google Sheets piece)
   - **Google Drive API** (for Google Drive piece)
   - **Google Calendar API** (for Google Calendar piece)
4. Navigate to **APIs & Services** → **OAuth consent screen**:
   - User Type: External (or Internal for Workspace)
   - App name, support email, developer contact
   - Add the following **Scopes** based on which pieces you need:

   Gmail Scopes:
   - `https://www.googleapis.com/auth/gmail.send` - Send emails
   - `https://www.googleapis.com/auth/gmail.readonly` - Read emails
   - `https://www.googleapis.com/auth/gmail.compose` - Compose emails
   - `email` - View email address

   Google Sheets Scopes:
   - `https://www.googleapis.com/auth/spreadsheets` - Full spreadsheet access
   - `https://www.googleapis.com/auth/drive.readonly` - View Drive files
   - `https://www.googleapis.com/auth/drive` - Full Drive access

   Google Drive Scopes:
   - `https://www.googleapis.com/auth/drive` - Full Drive access

   Google Calendar Scopes:
   - `https://www.googleapis.com/auth/calendar.events` - Manage calendar events
   - `https://www.googleapis.com/auth/calendar.readonly` - View calendars
5. Navigate to **APIs & Services** → **Credentials**
6. Click **Create Credentials** → **OAuth Client ID**
7. For OAuth Client ID:
   - Application type: **Web application**
   - Name: Your app name
   - Authorized redirect URIs:

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

8. Click **Create**
9. Copy the **Client ID** and **Client Secret**

> **Note:** You can use a single OAuth app for all Google pieces, or create separate ones for each.

HubSpot

Developer Portal: https://developers.hubspot.com/

Step 1: Access Legacy Apps

1. Go to HubSpot Developer Portal
2. In the left sidebar under "Development", click **Legacy Apps**
   - Or go directly to: https://developers.hubspot.com/my-apps
3. Click **Create app**

Step 2: Configure App Info

1. In the **App Info** tab:
   - **Public app name**: Enter your app name (e.g., "Activepieces Integration")
   - **App logo**: Optional - upload a square logo
   - **Description**: Optional - describe your app

Step 3: Configure Auth Settings

1. Click on the **Auth** tab
2. Scroll down to **Redirect URLs**
3. Click **+ Add redirect URL** and enter:

```
https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
```

Step 4: Add Required Scopes

> ⚠️ **Important: Only add scopes that Activepieces requests**
>
> HubSpot will reject the OAuth flow if your app has "Required" scopes that Activepieces doesn't request.
> Only add the scopes listed below as **Required**. Do NOT add extra scopes as "Required" - add them as "Optional" instead if needed.

1. Scroll down to the **Scopes** section
2. Click **+ Add new scope** and add **only** the following scopes as **Required**:

   CRM Scopes:
   - `crm.lists.read` - Read lists
   - `crm.lists.write` - Write lists
   - `crm.objects.companies.read` - Read companies
   - `crm.objects.companies.write` - Write companies
   - `crm.objects.contacts.read` - Read contacts
   - `crm.objects.contacts.write` - Write contacts
   - `crm.objects.custom.read` - Read custom objects
   - `crm.objects.custom.write` - Write custom objects
   - `crm.objects.deals.read` - Read deals
   - `crm.objects.deals.write` - Write deals
   - `crm.objects.line_items.read` - Read line items
   - `crm.objects.owners.read` - Read owners
   - `crm.objects.leads.read` - Read leads
   - `crm.objects.leads.write` - Write leads
   - `crm.schemas.companies.read` - Read company schemas
   - `crm.schemas.contacts.read` - Read contact schemas
   - `crm.schemas.custom.read` - Read custom schemas
   - `crm.schemas.deals.read` - Read deal schemas
   - `crm.schemas.line_items.read` - Read line item schemas

   Feature Scopes:
   - `automation` - Automation access
   - `content` - Content access
   - `e-commerce` - E-commerce access
   - `files` - Files access

- `forms` - Forms access
- `scheduler.meetings.meeting-link.read` - Read meeting links
- `settings.currencies.read` - Read currencies
- `settings.users.read` - Read users
- `settings.users.teams.read` - Read teams
- `tickets` - Tickets access

> **Note:** Do NOT add these scopes as "Required" (they cause OAuth failures):
>
> - `automation.sequences.read`
> - `automation.sequences.enrollments.write`
> - `external_integrations.forms.access`
> - `files.ui_hidden.read`
> - `forms-uploaded-files`
> - `oauth`
>
> If you need these, add them as **Optional scopes** instead.

Step 5: Create the App

1. Click **Create app** button at the bottom left
2. You may see a notice about "Subscribe to contact privacy deletion events" - this is **optional** and can be dismissed (it's for GDPR compliance)

Step 6: Get Your Credentials

1. After the app is created, go to the **Auth** tab
2. Under **App credentials**, find:
   - **App ID**: Your app's unique identifier
   - **Client ID**: Copy this value
   - **Client secret**: Click **Show**, then **Copy**

Step 7: Configure in Activepieces

1. In Activepieces Admin → **Cloud OAuth Apps**
2. Click **Add OAuth App**
3. Select **HubSpot** (or enter `@activepieces/piece-hubspot`)
4. Paste the **Client ID**
5. Paste the **Client Secret**
6. Save

Troubleshooting HubSpot OAuth

"Couldn't complete the connection - missing scopes" Error

This error occurs when your HubSpot app has **Required scopes** that Activepieces doesn't request. The error message will list the problematic scopes.

Solution:

1. Go to your HubSpot app → **Auth** tab → **Scopes**
2. Find the scopes mentioned in the error (e.g., `files.ui_hidden.read`, `automation.sequences.read`)
3. Either **Delete** these scopes, or change them from "Required" to "Optional"
4. Save and try connecting again

Common scopes that cause this error:

- `files.ui_hidden.read`
- `external_integrations.forms.access`
- `forms-uploaded-files`
- `automation.sequences.enrollments.write`
- `automation.sequences.read`

"Invalid redirect URI" Error

- Ensure the redirect URL in HubSpot exactly matches your Activepieces URL
- Check for trailing slashes - they must match exactly

App shows as "Draft"

- Draft apps work fine for OAuth - you don't need to publish to the marketplace
- Publishing is only required if you want to distribute the app publicly

Microsoft (Outlook, Teams, OneDrive)

Azure Portal: https://portal.azure.com/

1. Go to Azure Portal
2. Navigate to **Azure Active Directory** → **App registrations**
3. Click **New registration**
4. Configure:
   - Name: Your app name
   - Supported account types: Choose based on your needs (usually "Accounts in any organizational directory and personal Microsoft accounts")
   - Redirect URI: Web → `https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect`
5. Click **Register**
6. On the app overview page, copy the **Application (client) ID**
7. Navigate to **Certificates & secrets**
8. Click **New client secret**
9. Add description, select expiry, click **Add**
10. **Immediately copy the secret Value** (it won't be shown again)
11. Navigate to **API Permissions** → **Add a permission** → **Microsoft Graph** → **Delegated permissions**

Microsoft Outlook Scopes:

- `Mail.ReadWrite` - Read and write mail
- `Mail.Send` - Send mail
- `Calendars.Read` - Read calendars
- `offline_access` - Maintain access (refresh tokens)
- `User.Read` - Read user profile

Microsoft Teams Scopes:

- `openid` - Sign in
- `email` - View email address
- `profile` - View profile
- `offline_access` - Maintain access (refresh tokens)
- `User.Read` - Read user profile
- `Channel.Create` - Create channels
- `Channel.ReadBasic.All` - Read channel info
- `ChannelMessage.Send` - Send channel messages
- `ChannelMessage.Read.All` - Read channel messages
- `Team.ReadBasic.All` - Read team info
- `Chat.ReadWrite` - Read and write chats
- `TeamMember.Read.All` - Read team members
- `User.ReadBasic.All` - Read basic user info
- `Presence.Read.All` - Read user presence

Microsoft OneDrive Scopes:

- `Files.ReadWrite` - Read and write files
- `offline_access` - Maintain access (refresh tokens)

12. Click **Grant admin consent** if required for your organization

GitHub

Developer Settings: https://github.com/settings/developers

1. Go to GitHub Developer Settings
2. Click **OAuth Apps** → **New OAuth App**
3. Fill in:
   - Application name
   - Homepage URL: Your Activepieces URL
   - Authorization callback URL:

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

4. Click **Register application**
5. Copy the **Client ID**
6. Click **Generate a new client secret**
7. Copy the **Client Secret** immediately

Required Scopes (requested during OAuth flow):

- `admin:repo_hook` - Manage repository webhooks
- `admin:org` - Manage organization settings
- `repo` - Full repository access

GitLab

Applications Page: https://gitlab.com/-/profile/applications

1. Go to GitLab Applications
2. Click **Add new application**
3. Fill in:
   - Name: Your app name
   - Redirect URI:

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

   - Confidential: Check this box
   - **Scopes** (required by Activepieces):
     - `api` - Full API access
     - `read_user` - Read user info
4. Click **Save application**
5. Copy the **Application ID** (Client ID)
6. Copy the **Secret** (Client Secret)

Notion

Developer Portal: https://www.notion.so/my-integrations

1. Go to Notion Integrations
2. Click **New integration**
3. Fill in:
   - Name: Your integration name
   - Associated workspace: Select your workspace
   - Type: **Public** (for OAuth)
4. After creation, navigate to **OAuth & Permissions**

5. Add redirect URI:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Select required **Capabilities**:
   - Read content
   - Update content
   - Insert content
   - Read user information (including email)
7. Find credentials in **Secrets**:
   - **OAuth client ID**
   - **OAuth client secret**

> **Note:** Notion uses default scopes; no specific scope configuration is needed.

Dropbox

App Console: https://www.dropbox.com/developers/apps

1. Go to Dropbox App Console
2. Click **Create app**
3. Choose:
   - API: Scoped access
   - Access type: Full Dropbox (recommended) or App folder
   - Name: Your app name
4. Click **Create app**
5. In the **Settings** tab, add OAuth 2 redirect URI:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Navigate to **Permissions** tab and enable these scopes (required by Activepieces):
   - `files.metadata.write` - Edit file/folder metadata
   - `files.metadata.read` - View file/folder metadata
   - `files.content.write` - Edit file content
   - `files.content.read` - View file content
7. Click **Submit** to save permissions
8. Go back to **Settings** and find credentials:
   - **App key** (Client ID)
   - **App secret** (Client Secret)

Salesforce

Setup Location: Salesforce Setup → External Client App Manager

> ⚠️ **Important: Salesforce OAuth is Org-Specific**
>
> Unlike platforms like Slack where one OAuth app works for any workspace, **Salesforce OAuth apps are isolated to each org**. This means:
>
> | Aspect | Slack | Salesforce |
> |---|---|---|
> | App Registration | Centralized (api.slack.com) | Per-org (each Salesforce org) |
> | App Visibility | Any workspace can authorize | Only the org where app is created |
> | Multi-tenant Support | ✅ One app serves all users | ❌ Each org needs its own app |
>
> **What this means for you:**

> - If users will connect from **one Salesforce org**: Create the External Client App in that org
> - If users will connect from **multiple Salesforce orgs**: Each org admin must create their own External Client App with the same callback URL, OR publish your app on Salesforce AppExchange
> - The Consumer Key/Secret from Org A will **not work** for users logging in from Org B

Step 1: Create External Client App

1. Log into your Salesforce org as an admin
2. Go to **Setup** (gear icon → Setup)
3. In the Quick Find search box, type **"App Manager"**
4. Click on **External Client Apps** → **External Client App Manager**
5. Click **New External Client App** (top right)
6. Fill in the **Basic Information**:
   - **External Client App Name**: e.g., `Activepieces Integration`
   - **API Name**: Auto-generated (e.g., `Activepieces_Integration`)
   - **Contact Email**: Your email address
   - **Distribution State**: `Local`

Step 2: Configure OAuth Settings

1. After creating the app, go to the **Settings** tab
2. Expand **OAuth Settings** section
3. Click **Edit** if not already in edit mode
4. Set the **Callback URL**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

5. Under **OAuth Scopes**, move these from "Available" to "Selected":
   - `Manage user data via APIs (api)` - API access
   - `Full access (full)` - Complete Salesforce access
   - `Perform requests at any time (refresh_token, offline_access)` - For token refresh
6. Click **Save**

Step 3: Enable Authorization Code Flow

1. Go to the **Policies** tab
2. Expand **OAuth Policies** section
3. Click **Edit**
4. Under **Flow Enablement**, check:
   - ☑ **Enable Authorization Code and Credentials Flow**
   - Leave other flow options unchecked
5. Under **Plugin Policies**:
   - **Permitted Users**: Select `All users may self-authorize`
   - **IP Relaxation**: Select `Relax IP restrictions`
6. Click **Save**

Step 4: Get Consumer Credentials

1. Go back to the **Settings** tab
2. Under **OAuth Settings** → **App Settings**, click **Consumer Key and Secret**
3. You may need to verify your identity (Salesforce will send a verification code)
4. Copy:
   - **Consumer Key** (this is your Client ID)
   - **Consumer Secret** (this is your Client Secret)

Step 5: Configure in Activepieces

1. In Activepieces Admin → **Cloud OAuth Apps**
2. Add a new OAuth app for Salesforce
3. Enter the **Consumer Key** as Client ID

4. Enter the **Consumer Secret** as Client Secret
5. Save

Multi-Org Deployment

If you need to support users from multiple Salesforce orgs:

Option 1: Per-Org Setup (Recommended for small deployments)

- Each Salesforce org admin creates an External Client App in their org
- All apps use the same Callback URL (your Activepieces instance)
- Update Activepieces Cloud OAuth Apps with the credentials from the primary org
- Users from other orgs will need their org admin to share their Consumer Key/Secret

Option 2: AppExchange Listing (For large-scale deployments)

- Create a Salesforce Connected App and publish it on AppExchange
- Once approved, any Salesforce org can install your app
- This requires Salesforce security review and is more complex to set up

Option 3: Let Users Provide Their Own Credentials

- Don't configure Salesforce in Cloud OAuth Apps
- Users connect using custom credentials (they create their own Connected App)
- More work for end users but supports any org

Troubleshooting Salesforce OAuth

"OAUTH_EC_APP_NOT_FOUND" Error

- The Consumer Key doesn't match any External Client App in the Salesforce org
- **Most common cause**: User is logging into a different Salesforce org than where the app was created
- Verify you're using the correct Salesforce org (check the URL during OAuth)
- Wait 2-10 minutes after creating the app for it to propagate

"Authorization Code Flow not enabled" Error

- Go to the app's **Policies** tab → **OAuth Policies**
- Enable **"Enable Authorization Code and Credentials Flow"**

"Invalid redirect URI" Error

- Ensure the Callback URL exactly matches your Activepieces redirect URL
- Check for trailing slashes or protocol mismatches (https vs http)

User is redirected to wrong Salesforce org

- Clear browser cookies for salesforce.com domains
- Use an incognito/private browser window
- Or log out of all Salesforce orgs first, then log into the correct org

For Salesforce Sandbox

- Use `test.salesforce.com` instead of `login.salesforce.com`
- Create the External Client App in the sandbox org, not production
- Sandbox orgs are completely separate - credentials from production won't work

Snowflake

Setup Location: Snowflake Console → SQL Worksheet

> ⚠️ **Important: Snowflake OAuth is Account-Specific**

> Similar to Salesforce, **Snowflake OAuth integrations are isolated to each account**. This means:
>
> | Aspect | Centralized (e.g., Slack) | Snowflake |
> |---|---|---|
> | App Registration | Central developer portal | Per-account (SQL command in each account) |
> | App Visibility | Any workspace can authorize | Only the account where integration is created |
> | Multi-tenant Support | ✅ One app serves all users | ❌ Each account needs its own security integration |
>
> **What this means for you:**
>
> - If users connect from **one Snowflake account**: Create the security integration in that account
> - If users connect from **multiple accounts**: Each account admin must create their own security integration with the same redirect URI
> - The Client ID/Secret from Account A will **not work** for users logging in from Account B

Prerequisites

- **ACCOUNTADMIN role** (or a role with `CREATE INTEGRATION` privilege)
- Access to a **SQL Worksheet** in Snowflake console
- Your **Account Identifier** (found in account menu → View account details)

Step 1: Create Security Integration

Open a **SQL Worksheet** in your Snowflake console and run:

```
CREATE SECURITY INTEGRATION "activepieces"
  TYPE = OAUTH
  OAUTH_CLIENT = CUSTOM
  OAUTH_CLIENT_TYPE = 'CONFIDENTIAL'
  OAUTH_REDIRECT_URI = 'https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect'
  ENABLED = TRUE
  OAUTH_ISSUE_REFRESH_TOKENS = TRUE;
```

> **Note:** Replace the redirect URI with your actual Activepieces redirect URL.

Step 2: Retrieve Client Credentials

Run the following query to get your **Client ID** and **Client Secret**:

```
SELECT SYSTEM$SHOW_OAUTH_CLIENT_SECRETS('activepieces');
```

The result is a JSON object. Copy:

- `OAUTH_CLIENT_ID` → This is your **Client ID**
- `OAUTH_CLIENT_SECRET` → This is your **Client Secret**

Step 3: Find Your Account Identifier

1. Click the **account icon** at the bottom-left of the Snowflake console
2. Click **View account details**
3. Copy the **Account Identifier** (e.g., `xy12345.us-east-1` or `orgname-accountname`)

Step 4: Configure in Activepieces

1. In Activepieces Admin → **Cloud OAuth Apps**
2. Add a new OAuth app for Snowflake
3. Enter the **Client ID** from Step 2
4. Enter the **Client Secret** from Step 2
5. Save

When users connect, they will need to provide:

- **Account Identifier** (from Step 3)
- Optional: Default Database, Warehouse, and Role

Multi-Account Deployment

If you need to support users from multiple Snowflake accounts:

Option 1: Per-Account Setup (Recommended)

- Each Snowflake account admin runs the `CREATE SECURITY INTEGRATION` SQL
- All integrations use the same redirect URI (your Activepieces instance)
- Users from each account can connect using their account's credentials

Option 2: Let Users Provide Their Own Credentials

- Don't configure Snowflake in Cloud OAuth Apps
- Users create their own security integration in their Snowflake account
- Users then use the "Username & Password / Key Pair" authentication method instead of OAuth

Option 3: Use Non-OAuth Authentication

- Snowflake supports Username/Password and Key Pair authentication
- These methods don't require a security integration
- Better for multi-tenant scenarios where OAuth setup isn't practical

Troubleshooting Snowflake OAuth

"Invalid client" or "Integration not found" Error

- Verify the security integration exists: `SHOW INTEGRATIONS;`
- Ensure the integration is enabled: `ALTER SECURITY INTEGRATION "activepieces" SET ENABLED = TRUE;`
- Wait 1-2 minutes after creating the integration for it to propagate

"Invalid redirect URI" Error

- The redirect URI in the security integration must exactly match your Activepieces URL
- Update with: `ALTER SECURITY INTEGRATION "activepieces" SET OAUTH_REDIRECT_URI = 'your-correct-url';`

"Insufficient privileges" Error

- Ensure you're using a role with adequate permissions
- The user connecting must have a role that can access the requested database/warehouse

Connection works but queries fail

- Check the Default Database, Warehouse, and Role settings
- Ensure the connected user has access to those resources

User logs in but gets redirected to wrong account

- Clear browser cookies for `snowflakecomputing.com`
- Ensure the Account Identifier matches the account where the integration was created
- Use incognito/private browsing to avoid cached sessions

Asana

Developer Console: https://app.asana.com/0/developer-console

1. Go to Asana Developer Console
2. Click **Create new app**
3. Fill in app name and accept terms
4. Navigate to **OAuth** section

5. Add redirect URL:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Find credentials:
   - **Client ID**
   - **Client Secret**

> **Note:** Activepieces uses the `default` scope which provides full API access.

ClickUp

Developer Portal: https://app.clickup.com/settings/integrations

1. Go to ClickUp Settings → **Integrations** → **ClickUp API**
2. Click **Create an App**
3. Fill in:
   - App Name
   - Redirect URL(s):

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

4. Click **Create App**
5. Copy:
   - **Client ID**
   - **Client Secret**

> **Note:** ClickUp uses default scopes; no specific scope configuration is needed.

Monday.com

Developer Center: https://monday.com/developers/apps

> **Note:** Monday.com piece uses **API Token authentication**, not OAuth2. Users provide their API token directly in Activepieces - no Cloud OAuth App configuration needed.

To get an API token:

1. Go to Monday.com
2. Click your avatar → **Developers** → **Developer Center**
3. Or go directly to **Admin** → **API**
4. Copy the **API v2 Token**

Trello

Power-Up Admin: https://trello.com/power-ups/admin

> **Note:** Trello piece uses **API Key + Token authentication**, not OAuth2. Users provide credentials directly in Activepieces - no Cloud OAuth App configuration needed.

To get API credentials:

1. Go to Trello Power-Up Admin
2. Click **New** to create a Power-Up
3. Fill in Power-Up details
4. Get your **API Key** from https://trello.com/app-key

5. Generate a **Token** using the link on the API key page

Jira

Developer Console: https://developer.atlassian.com/console/myapps/

> **Note:** Jira Cloud piece uses **Custom Auth (Email + API Token)**, not OAuth2. Users provide credentials directly in Activepieces - no Cloud OAuth App configuration needed.

To get API credentials:

1. Go to Atlassian Account Settings
2. Click **Create API token**
3. Give it a label and click **Create**
4. Copy the token
5. In Activepieces, provide:
   - **Email**: Your Atlassian account email
   - **API Token**: The token you just created
   - **Instance URL**: Your Jira instance (e.g., `https://yourcompany.atlassian.net`)

Linear

Settings: https://linear.app/settings/api

> **Note:** Linear piece uses **API Key authentication**, not OAuth2. Users provide their API key directly in Activepieces - no Cloud OAuth App configuration needed.

To get an API key:

1. Go to Linear API Settings
2. Under **Personal API keys**, click **Create key**
3. Give it a label and select permissions
4. Copy the generated API key

Figma

Developer Portal: https://www.figma.com/developers

1. Go to Figma Developers
2. Click **My Apps** → **Create a new app**
3. Fill in app details
4. Add **Callback URL**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

5. Submit for review if needed
6. Find credentials:
   - **Client ID**
   - **Client Secret**

Required OAuth Scopes (automatically requested):

- `file_read` - Read Figma files and projects

Zoom

Marketplace: https://marketplace.zoom.us/

1. Go to Zoom App Marketplace
2. Click **Develop** → **Build App**
3. Choose **OAuth** app type
4. Fill in app information
5. Add **Redirect URL for OAuth**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Add required **Scopes** based on features you need:
   - `meeting:read` - View meetings
   - `meeting:write` - Create/update meetings
   - `user:read` - View user info
   - `recording:read` - View recordings
7. Find credentials in **App Credentials**:
   - **Client ID**
   - **Client Secret**

> **Note:** Activepieces uses default scopes; add scopes based on the Zoom features you need.

Todoist

Developer Console: https://developer.todoist.com/appconsole.html

1. Go to Todoist App Console
2. Click **Create a new app**
3. Fill in app name
4. Add **OAuth redirect URL**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

5. Find credentials:
   - **Client ID**
   - **Client Secret**

Required OAuth Scopes (automatically requested):

- `data:read_write` - Full read and write access to tasks, projects, labels, and comments

Typeform

Developer Portal: https://admin.typeform.com/workspaces

1. Go to Typeform Admin → **Developer apps**
2. Click **Register a new app**
3. Fill in:
   - Application name
   - Redirect URI(s):

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

4. Find credentials:
   - **Client ID**
   - **Client Secret**

Required OAuth Scopes (automatically requested):

- `webhooks:write` - Create and manage webhooks for form responses
- `forms:read` - Read form definitions and responses

QuickBooks

Developer Portal: https://developer.intuit.com/

1. Go to Intuit Developer Portal
2. Sign in and go to **Dashboard**
3. Click **Create an app**
4. Select **QuickBooks Online and Payments**
5. Fill in app details
6. Navigate to **Keys & OAuth**
7. Add **Redirect URIs**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

8. Select the following **Scopes** (required by Activepieces):
   - `com.intuit.quickbooks.accounting` - Full QuickBooks accounting access
9. Find credentials:
   - **Client ID**
   - **Client Secret**

Intercom

Developer Hub: https://developers.intercom.com/

1. Go to Intercom Developer Hub
2. Create a new app or select existing
3. Navigate to **Authentication**
4. Enable **OAuth**
5. Add **Redirect URLs**:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
   ```

6. Find credentials:
   - **Client ID**
   - **Client Secret**

> **Note:** Activepieces uses default Intercom scopes which provide full access to the Intercom API based on your app's permissions.

Zendesk

Developer Portal: https://developer.zendesk.com/

> **Note:** Zendesk piece uses **Custom Auth (Email + API Token)**, not OAuth2. Users provide credentials directly in Activepieces - no Cloud OAuth App configuration needed.

To get API credentials:

1. Log into your Zendesk Admin Center
2. Go to **Admin** → **Channels** → **API**
3. Enable **Token Access**
4. Click **Add API Token**

5. Give it a description and click **Create**
6. Copy the token immediately (it won't be shown again)
7. In Activepieces, provide:
   - **Email**: Your Zendesk agent email (append `/token` - e.g., `agent@company.com/token`)
   - **Token**: The API token you just created
   - **Subdomain**: Your Zendesk subdomain (e.g., `yourcompany` from `yourcompany.zendesk.com`)

Mailchimp

Developer Portal: https://mailchimp.com/developer/

1. Go to Mailchimp Developer
2. Click **Register an App**
3. Fill in:
   - App name
   - Redirect URI:

     ```
     https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect
     ```

4. Find credentials:
   - **Client ID**
   - **Client Secret**

> **Note:** Activepieces uses default Mailchimp scopes which provide full access to your Mailchimp account based on your API permissions.

Shopify

Partner Dashboard: https://partners.shopify.com/

> **Note:** Shopify piece uses **Custom Auth (Shop Name + Admin Access Token)**, not OAuth2. Users provide credentials directly in Activepieces - no Cloud OAuth App configuration needed.

To get API credentials:

1. Go to your Shopify Admin → **Settings** → **Apps and sales channels**
2. Click **Develop apps** → **Create an app**
3. Configure **Admin API scopes** based on what you need:
   - `read_products`, `write_products` - Product management
   - `read_orders`, `write_orders` - Order management
   - `read_customers`, `write_customers` - Customer management
   - Add other scopes as needed
4. Click **Install app**
5. Copy the **Admin API access token** (shown only once)
6. In Activepieces, provide:
   - **Shop Name**: Your shop's myshopify.com subdomain (e.g., `mystore` from `mystore.myshopify.com`)
   - **Admin Access Token**: The token you just copied

Stripe

Dashboard: https://dashboard.stripe.com/

> **Note:** Stripe piece uses **API Secret Key authentication**, not OAuth2. Users provide their API key directly in Activepieces - no Cloud OAuth App configuration needed.

To get API credentials:

1. Go to Stripe Dashboard
2. Navigate to **Developers** → **API keys**
3. Copy the **Secret key** (starts with `sk_live_` or `sk_test_`)
4. In Activepieces, provide the Secret Key

For testing, use your test mode API key (`sk_test_...`).

Configuring in Activepieces

Once you have your OAuth credentials:

1. Log in to Activepieces as a **Super Admin**
2. Navigate to **Super Admin Dashboard** → **Cloud OAuth Apps**
3. Click **Add OAuth App**
4. Select the **Piece** from the dropdown
5. Enter the **Client ID**
6. Enter the **Client Secret**
7. Click **Add**

The piece will now show a simplified "Connect" button for users instead of requiring them to enter credentials manually.

App Webhook Configuration (For Triggers)

Some pieces use **App Webhooks** for their triggers. These require additional configuration beyond OAuth to receive real-time events from the service.

Pieces Requiring App Webhook Configuration

Piece	Triggers	Webhook URL	Signing Secret Location
Slack	14 triggers (new message, reaction, mention, etc.)	`/api/v1/app-events/slack`	Slack App → Basic Info → Signing Secret
Intercom	18 triggers (conversation, lead, user, etc.)	`/api/v1/app-events/intercom`	Intercom App → Webhooks → Hub Secret
Square	Payment and order events	`/api/v1/app-events/square`	Square App → Webhooks → Signature Key
Facebook Leads	New lead trigger	`/api/v1/app-events/facebook-leads`	Facebook App → App Secret

Step 1: Configure Webhook Signing Secret

Add the signing secret to your Activepieces environment variables:

Single piece:

```
AP_APP_WEBHOOK_SECRETS='{"@activepieces/piece-slack":{"webhookSecret":"YOUR_SLACK_SIGNING_SECRET"}}'
```

Multiple pieces:

```
AP_APP_WEBHOOK_SECRETS='{"@activepieces/piece-slack":{"webhookSecret":"SLACK_SECRET"},"@activepieces/piece-square":{"webhookSecret":'
```

Step 2: Configure Event Subscriptions in Provider

Slack Event Subscriptions

1. Go to your Slack App → **Event Subscriptions**
2. Enable Events
3. Set **Request URL** to:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/api/v1/app-events/slack
   ```

4. Wait for Slack to verify the URL (should show ✓ Verified)
5. Subscribe to bot events:
   - `message.channels` - Messages in public channels
   - `message.im` - Direct messages
   - `message.groups` - Messages in private channels
   - `message.mpim` - Messages in group DMs
   - `reaction_added` - Reactions added to messages
   - `app_mention` - Bot mentions
   - `channel_created` - New channel created
   - `team_join` - New user joined
   - `emoji_changed` - Custom emoji added

Intercom Webhooks

1. Go to Intercom Developer Hub → **Webhooks**
2. Add webhook URL:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/api/v1/app-events/intercom
   ```

3. Select relevant topics (conversation, lead, user events)

Square Webhooks

1. Go to Square Developer Dashboard → **Webhooks**
2. Add webhook URL:

   ```
   https://overdiversely-preeruptive-margaretta.ngrok-free.dev/api/v1/app-events/square
   ```

3. Select event types (payments, orders, etc.)

Important Notes

- **This configuration is required for BOTH development and production**
- Without the webhook signing secret, triggers will fail to verify incoming webhooks
- The webhook URL must be publicly accessible (use ngrok for local development)
- Each self-hosted instance needs its own webhook configuration

Troubleshooting

Common Issues

"Invalid redirect URI" error

- Ensure the redirect URL in the OAuth provider exactly matches: `https://overdiversely-preeruptive-margaretta.ngrok-free.dev/redirect`
- Check for trailing slashes
- Ensure HTTPS is used (not HTTP)

"Invalid client" error

- Double-check the Client ID and Client Secret

- Ensure there are no leading/trailing spaces
- Regenerate the secret if needed

Scopes/permissions issues

- Ensure you've enabled the required API scopes in the OAuth provider
- Some providers require app verification for certain scopes

Token refresh failures

- Ensure "offline_access" or equivalent refresh token scope is enabled
- Check if the OAuth app is still active in the provider's dashboard

Getting Help

If you encounter issues:

1. Check the OAuth provider's documentation
2. Review the Activepieces logs for detailed error messages
3. Ensure your Activepieces instance has network access to the OAuth provider

Supported Pieces Reference

OAuth2 Pieces (Cloud OAuth App Configurable)

Any piece that uses OAuth2 authentication can be configured in the Cloud OAuth Apps admin panel. The table below lists commonly used OAuth2 pieces, but the system is **not limited to this list** - you can add any OAuth2 piece by entering its piece name (e.g., `@activepieces/piece-snowflake`), Client ID, and Client Secret.

> **Legend:** = Org/Account-Specific OAuth (requires per-org setup)

Piece Name	Provider	OAuth Model	Key Scopes	Developer Portal
`@activepieces/piece-slack`	Slack	Centralized	`channels:read`, `chat:write`, `users:read` + 30 more	https://api.slack.com/apps
`@activepieces/piece-gmail`	Google	Centralized	`gmail.send`, `gmail.readonly`, `gmail.compose`, `email`	https://console.cloud.google.cor
`@activepieces/piece-google-sheets`	Google	Centralized	`spreadsheets`, `drive.readonly`, `drive`	https://console.cloud.google.cor
`@activepieces/piece-google-drive`	Google	Centralized	`drive` (full access)	https://console.cloud.google.cor
`@activepieces/piece-google-calendar`	Google	Centralized	`calendar.events`, `calendar.readonly`	https://console.cloud.google.cor
`@activepieces/piece-hubspot`	HubSpot	Centralized	`crm.objects.*`, `crm.lists.*`, `automation` + more	https://developers.hubspot.com/
`@activepieces/piece-microsoft-outlook`	Microsoft	Centralized	`Mail.ReadWrite`, `Mail.Send`, `Calendars.Read`	https://portal.azure.com/
`@activepieces/piece-microsoft-teams`	Microsoft	Centralized	`Channel.*`, `Chat.ReadWrite`, `Team.ReadBasic.All`	https://portal.azure.com/
`@activepieces/piece-microsoft-onedrive`	Microsoft	Centralized	`Files.ReadWrite`, `offline_access`	https://portal.azure.com/
`@activepieces/piece-github`	GitHub	Centralized	`admin:repo_hook`, `admin:org`, `repo`	https://github.com/settings/deve
`@activepieces/piece-gitlab`	GitLab	Centralized	`api`, `read_user`	https://gitlab.com/-/profile/applic

Piece Name	Provider	OAuth Model	Key Scopes	Developer Portal
`@activepieces/piece-notion`	Notion	Centralized	Default scopes (full integration access)	https://www.notion.so/my-integr
`@activepieces/piece-dropbox`	Dropbox	Centralized	`files.metadata.*` , `files.content.*`	https://www.dropbox.com/devel
`@activepieces/piece-salesforce`	Salesforce	🏢 Org-Specific	`refresh_token` , `full` , `api`	Salesforce Setup → App Manag
`@activepieces/piece-snowflake`	Snowflake	🏢 Account-Specific	`session:role-any` , `refresh_token`	Snowflake SQL Worksheet
`@activepieces/piece-asana`	Asana	Centralized	`default` (full API access)	https://app.asana.com/0/develo
`@activepieces/piece-clickup`	ClickUp	Centralized	Default scopes	https://app.clickup.com/settings
`@activepieces/piece-figma`	Figma	Centralized	`file_read`	https://www.figma.com/develop
`@activepieces/piece-zoom`	Zoom	Centralized	Default scopes	https://marketplace.zoom.us/
`@activepieces/piece-todoist`	Todoist	Centralized	`data:read_write`	https://developer.todoist.com/ap
`@activepieces/piece-typeform`	Typeform	Centralized	`webhooks:write` , `forms:read`	https://admin.typeform.com/
`@activepieces/piece-quickbooks`	Intuit	Centralized	`com.intuit.quickbooks.accounting`	https://developer.intuit.com/
`@activepieces/piece-intercom`	Intercom	Centralized	Default scopes	https://developers.intercom.com
`@activepieces/piece-mailchimp`	Mailchimp	Centralized	Default scopes	https://mailchimp.com/develope

Org/Account-Specific OAuth Pieces

These pieces require special attention for multi-tenant deployments:

Piece	Why Org-Specific	Setup Complexity	Multi-Tenant Strategy
Salesforce	OAuth apps are created per Salesforce org in Setup → External Client App Manager	Medium	Per-org setup, AppExchange listing, or user-provided credentials
Snowflake	Security integrations are created per Snowflake account via SQL	Medium	Per-account setup or use Username/Password/Key Pair auth instead

See the individual Salesforce and Snowflake sections for detailed setup instructions.

Non-OAuth2 Pieces (API Key / Custom Auth)

These pieces use API keys or custom authentication methods. Users provide credentials directly in Activepieces - no Cloud OAuth App configuration needed:

Piece Name	Provider	Auth Type	Credentials Portal
`@activepieces/piece-monday`	Monday.com	API Token	https://monday.com/ → Admin → API
`@activepieces/piece-trello`	Trello	API Key + Token	https://trello.com/app-key
`@activepieces/piece-jira-cloud`	Atlassian	Email + API Token	https://id.atlassian.com/manage-profile/security/api-tokens
`@activepieces/piece-linear`	Linear	API Key	https://linear.app/settings/api
`@activepieces/piece-zendesk`	Zendesk	Email + API Token	Zendesk Admin → Channels → API
`@activepieces/piece-shopify`	Shopify	Admin Access Token	Shopify Admin → Settings → Apps → Develop apps

Piece Name	Provider	Auth Type	Credentials Portal
`@activepieces/piece-stripe`	Stripe	Secret API Key	https://dashboard.stripe.com/apikeys